AuroraStay QnA System Deployment Guide

1. What This Guide Covers

The notebook is divided into several major phases. The **earlier phases** (data loading with Docling, text chunking, embedding with a fine-tuned HuggingFace model, FAISS vector storage, prompt-based generation via OpenAI, and a four-metric evaluation pipeline using DeepEval) all happen **inside the notebook**. Those phases produce two key artefacts:

- **A serialized FAISS vector index** containing the pre-computed embeddings for every text chunk.
- **A set of proven RAG utility functions** (retrieve, generate, and the full pipeline) that have already been tested and evaluated.

The **Deployment sections** take those artefacts and wrap them in production-ready files so the system can run *outside* the notebook as an independent web application. This guide walks through every deployment file, explains why it exists, what it does, and how all the pieces connect.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

2. High-Level Deployment Architecture

The deployment pipeline has four layers. Each layer feeds into the next, moving the system from a notebook prototype to a live, containerized service.

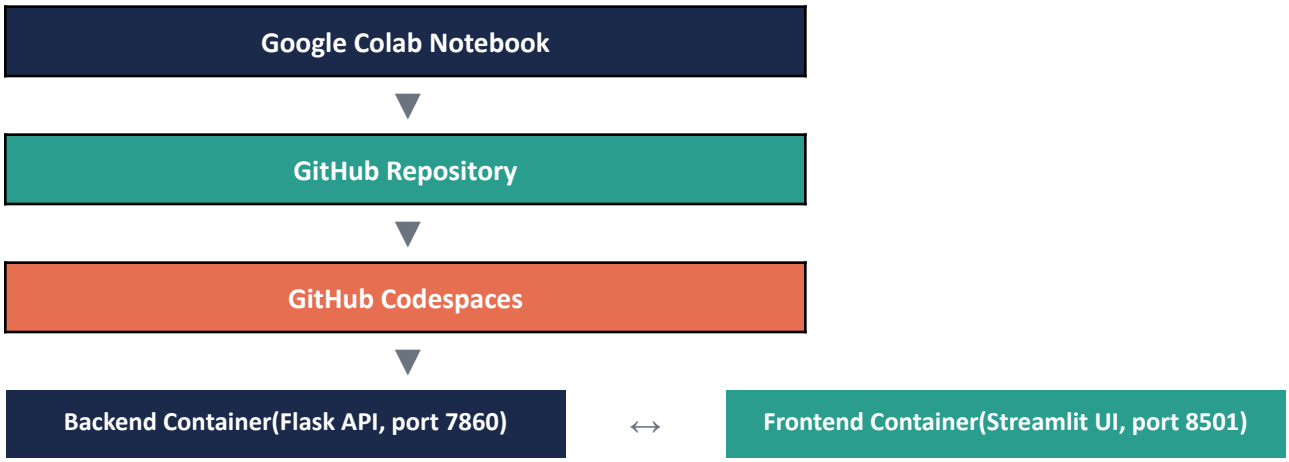


Figure 1. The four-layer deployment pipeline. The notebook generates files, pushes them to GitHub, and Codespaces builds and runs two Docker containers.

Layer-by-Layer Explanation

Google Colab Notebook. This is the development environment. All data exploration, chunking strategy selection, embedding model loading, vector-store creation, prompt engineering, and evaluation happen here. Once the evaluation metrics meet the success criteria (composite score above the threshold), the notebook switches to deployment mode: it writes out every file the production system needs using the %%writefile magic command and serializes the FAISS index to disk.

GitHub Repository. Acts as the bridge between the notebook and the hosting environment. The notebook uses the PyGithub library to authenticate via a Personal Access Token (stored in Colab Secrets, never hard-coded) and pushes all deployment files into a public repo with two top-level folders: backend/ and frontend/. This clean separation makes it easy for Codespaces to build each container independently.

GitHub Codespaces. A cloud-hosted development environment that can build Docker images and run containers without any local machine setup. It automatically detects the Dockerfiles in the repo, builds the images, and starts the containers. Codespaces also provides port forwarding, which means the running services get a public URL that the notebook (or any browser) can reach.

Backend and Frontend Containers. The final product. The backend container loads the FAISS index and the embedding model once at startup, then exposes a Flask REST API on port 7860. The frontend container runs a Streamlit web app on port 8501 that provides a graphical interface for hotel staff to type questions and see answers. The two containers communicate over HTTP within the Codespace network.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

3. Backend: What Each File Does and Why

The backend folder contains five items. Together they turn the RAG pipeline from a notebook experiment into a self-contained web service.

faiss_index/	Pre-built vector embeddings
rag_utils.py	Core RAG functions (retrieve, generate, pipeline)
app.py (Flask)	REST API endpoints for the frontend
requirements.txt	Pinned Python dependencies
Dockerfile	Container build recipe

Figure 2. Each backend file and its role.

faiss_index/ (Serialized Vector Store)

Before any other deployment file is created, the notebook calls `vectorstore.save_local("backend_files/faiss_index")` to write the FAISS index to disk. This folder contains all the pre-computed embedding vectors plus the metadata that links each vector back to its source PDF.

Why does this matter?

Without the serialized index, the backend would have to re-embed every text chunk on every startup. That process takes several minutes and requires the original PDF files, the Docling converter, and the chunking pipeline to all be present inside the container. By saving the index once, the backend can load it in seconds and needs none of those extra components.

rag_utils.py (RAG Pipeline Logic)

This file contains three functions that are the heart of the RAG system:

- **retrieve(query, k, vectorstore)** runs a FAISS similarity search and returns the top-k chunks, each formatted with its text content and source-document metadata.
- **generate(query, retrieved_chunks, model)** builds a structured prompt that includes the user's question, the retrieved passages, and a strict set of ten answer rules (use only retrieved info, no outside knowledge, mention conflicts, etc.) and sends it to the LLM.
- **rag(query, k, model_name, temperature, top_p, max_tokens, vectorstore)** ties retrieval and generation into a single callable pipeline. It creates a fresh LLM instance with the specified parameters, retrieves chunks, generates an answer, and returns both.

These functions are identical to the ones tested in the notebook, with one key difference: API credentials are read from environment variables (`os.getenv('OPENAI_API_KEY')`) instead of a config file, because the deployed environment uses GitHub Codespace secrets rather than a local JSON file. This keeps sensitive keys out of the source code.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

app.py (Flask REST API)

The Flask application is the web-server layer that makes the RAG pipeline accessible over HTTP. At startup it does two heavy operations:

- Loads the fine-tuned embedding model (`HarishMaths/Hotel-Policy-Embedding`) onto the CPU.
- Loads the saved FAISS index from disk using `FAISS.load_local()` with `allow_dangerous_deserialization=True` (required because FAISS uses pickle internally).

Loading these once at startup (rather than on every request) is critical for performance. The embedding model alone takes several seconds to initialize, and the FAISS index occupies significant memory. Reloading them on every request would make the service unusably slow.

The app exposes three POST endpoints:

Endpoint	Input	Output
/v1/relevant_chunks	query, k	Top-k retrieved chunks only
/v1/answer_with_relevant_chunks	query, k, model_name, temperature, top_p, max_tokens	Generated answer + supporting chunks
/v1/answer	query, k, model_name, temperature, top_p, max_tokens	Generated answer only

Table 1. The three REST endpoints and their inputs/outputs.

Having separate endpoints lets the frontend offer different modes: staff can choose to see just the answer, or the answer together with the source chunks so they can verify the information before sharing it with a guest.

requirements.txt (Backend Dependencies)

This file pins every Python package the backend needs:

Package	Version	Purpose
flask	3.1.3	Web framework for the REST API
gunicorn	26.2.0	Production-grade WSGI server
langchain-huggingface	1.2.2	Load fine-tuned HuggingFace embedding model
langchain-community	0.4.2	FAISS integration via LangChain
langchain-openai	1.6.1	Call OpenAI LLM through LangChain
sentence-transformers	6.1.0	Underlying model framework
faiss-cpu	1.15.0	CPU build of the FAISS vector search library

Table 2. Key backend dependencies and why each is needed.

Pinning exact versions is important because it guarantees that the container always installs the same software stack, regardless of when or where it is built. Without pinning, a newer (and potentially incompatible) version of a library could silently break the system.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

4. The Backend Dockerfile, Step by Step

A Dockerfile is a plain-text recipe that tells Docker exactly how to build a container image. Think of the image as a frozen snapshot of a fully configured machine: it includes the operating system, all libraries, the application code, the model weights, and the vector index. Once built, the image can be started anywhere Docker runs.

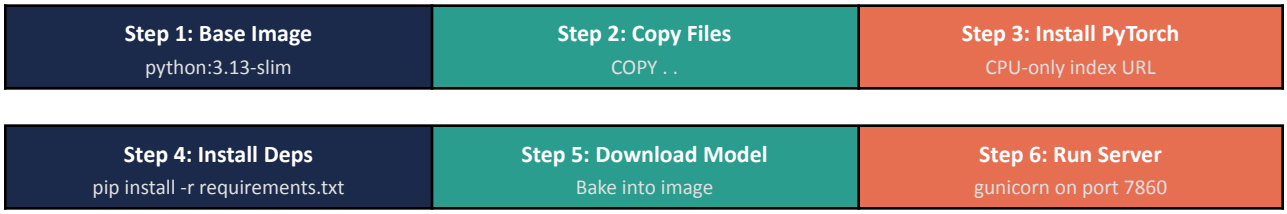


Figure 3. The six build stages of the backend Dockerfile.

Why Each Step Matters

Step 1 - Base Image (python:3.13-slim). The 'slim' variant strips out documentation and rarely-used system packages, keeping the starting image as small as possible. A smaller image means faster builds, faster downloads, and lower resource consumption in Codespaces.

Step 2 - Copy Files. `COPY . .` transfers everything in the backend/ folder (app.py, rag_utils.py, requirements.txt, and the entire faiss_index directory) into the container's /app working directory. From this point on, the container has all the code and data it needs.

Step 3 - Install PyTorch (CPU-only). PyTorch is a dependency of sentence-transformers. The default PyTorch package bundles CUDA GPU drivers, adding several gigabytes. Since Codespaces runs on CPU, installing from the special `https://download.pytorch.org/whl/cpu` index gives us a version that is a fraction of the size and perfectly adequate for inference.

Step 4 - Install Dependencies. Installs everything listed in requirements.txt. PyTorch was installed separately in Step 3 because it needs a special index URL; all remaining packages come from the default PyPI registry.

Step 5 - Download Embedding Model at Build Time. A `RUN python -c "..."` command instantiates the embedding model once, which forces HuggingFace to download and cache the weights inside the image. The environment variables `TRANSFORMERS_OFFLINE=1` and `HF_HUB_OFFLINE=1` are then set so the container never contacts Hugging Face at runtime. This makes startup faster and removes an external network dependency.

Step 6 - Run Server (Gunicorn). Flask ships with a lightweight development server that is not designed for production traffic. Gunicorn is a production-grade WSGI server that handles concurrent requests properly. The flags `-w 1` (one worker) and `--timeout 120` (two-minute timeout) are chosen because the embedding model is memory-heavy and RAG queries can take up to a minute when the LLM is generating a response.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

5. Frontend: What Each File Does and Why

The frontend folder is deliberately simple. Its only job is to provide a user-friendly web interface that sends HTTP requests to the backend and displays the results.

app.py (Streamlit)	▸	User interface for hotel staff
requirements.txt	▸	Only 2 packages: streamlit, requests
Dockerfile	▸	Lightweight container build recipe

Figure 4. Each frontend file and its role.

app.py (Streamlit UI)

The Streamlit application provides a clean, browser-based interface with four main elements:

- **A sidebar** with sliders for k (number of chunks to retrieve), temperature (LLM creativity), and max tokens (answer length). These map directly to the parameters the Flask API accepts.
- **A radio button** to choose between 'Answer only' mode (calls /v1/answer) and 'Answer + Relevant Chunks' mode (calls /v1/answer_with_relevant_chunks).
- **A text input** for the user's question and a Submit button.
- **Error handling** for both HTTP errors (non-200 status) and connection failures (backend not running).

When the user clicks Submit, the app packages the query and all slider values into a JSON payload, sends a POST request to the appropriate backend endpoint, and renders the answer. In 'Answer + Relevant Chunks' mode, each supporting chunk is displayed inside an expandable section so staff can verify the source before sharing it with a guest.

Dockerfile (Frontend)

The frontend Dockerfile is much simpler than the backend one:

- Same slim Python base image.
- Copies files and installs only two packages (streamlit, requests).
- Runs Streamlit on port 8501, bound to `0.0.0.0` so it is reachable outside the container.
- Disables XSRF protection (`--server.enableXsrfProtection=false`) to allow access through Codespace port forwarding, which changes the origin header.

Because the frontend has no heavy models or large data files, its image builds much faster and uses far less memory than the backend image.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

6. Pushing Files to GitHub

After all deployment files have been written to the local `backend_files/` and `frontend_files/` directories, the notebook pushes them to GitHub programmatically using the PyGithub library.

How the Push Works

- The GitHub Personal Access Token is retrieved from Colab Secrets (never hard-coded in the notebook).
- The notebook checks whether the repository already exists. If not, it creates a new public repo with `user.create_repo()`.
- For each file, it checks whether the file already exists in the repo. If yes, it updates the file; if no, it creates it. This makes the push idempotent: you can re-run the cell without errors.
- Binary files (the FAISS index uses pickle) are handled automatically by PyGithub's encoding layer.

Repository Path	Local Source	Description
backend/app.py	backend_files/app.py	Flask REST API
backend/rag_utils.py	backend_files/rag_utils.py	RAG pipeline functions
backend/requirements.txt	backend_files/requirements.txt	Pinned backend dependencies
backend/Dockerfile	backend_files/Dockerfile	Backend container build recipe
backend/faiss_index/*	backend_files/faiss_index/*	Serialized FAISS vector store (binary)
frontend/app.py	frontend_files/app.py	Streamlit UI application
frontend/requirements.txt	frontend_files/requirements.txt	Frontend dependencies (2 packages)
frontend/Dockerfile	frontend_files/Dockerfile	Frontend container build recipe

Table 3. Every file pushed from the notebook to the GitHub repository.

Codespace Secrets

After the push, you must add two secrets to GitHub Codespaces:

- **OPENAI_API_KEY**: the API key that lets the backend call the OpenAI LLM.
- **OPENAI_API_BASE**: the base URL for the OpenAI-compatible API endpoint.

Codespaces injects these as environment variables inside the container. This is why `rag_utils.py` uses `os.getenv('OPENAI_API_KEY')` rather than reading a config file. Secrets never appear in source code, commit history, or container logs.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

7. End-to-End Request Flow

Once both containers are running inside Codespaces, a single user query travels through the following path:

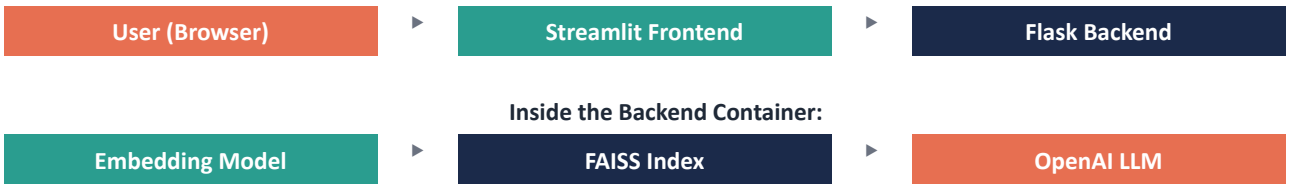


Figure 5. Request flow from user query to generated answer. The top row shows the user-visible path; the bottom row shows what happens inside the backend container.

Step-by-Step Walkthrough

- **1.** The hotel staff member opens the Streamlit URL in a browser, types a natural-language question (e.g., 'What is the pet charge at ASH-CHI Loop?'), adjusts parameters using the sidebar sliders, and clicks Submit.
- **2.** The Streamlit app assembles a JSON payload containing the query, k, temperature, top_p, and max_tokens, then sends a POST request to the selected backend endpoint.
- **3.** The Flask API receives the JSON payload. The rag() function (from rag_utils.py) is called.
- **4.** Inside rag(), the retrieve() function passes the query to the embedding model, which converts it into a vector. FAISS then compares that vector against all stored chunk vectors and returns the top-k most similar chunks, complete with their source-document metadata.
- **5.** The generate() function builds a structured prompt: the user's question, the retrieved passages, and ten strict answer rules (e.g., use only retrieved information, mention conflicts, say 'not enough information' if needed).
- **6.** The assembled prompt is sent to the OpenAI LLM (gpt-4o-mini by default). The LLM generates a grounded, policy-faithful answer.
- **7.** The Flask API packages the answer, the supporting chunks, and all parameter values into a JSON response and sends it back to the Streamlit frontend.
- **8.** The Streamlit app displays the answer. If the user chose 'Answer + Relevant Chunks' mode, each chunk appears in a collapsible expander so staff can verify which policy document the information came from before sharing it with a guest.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

8. Testing the Deployed API

The final Deployment sections in the notebook verify that the live service works correctly by treating the Codespace backend as a remote API. Two tests are run:

Test	Endpoint	Payload	Expected Output
Retrieval only	/v1/relevant_chunks	query, k=3	JSON with the original query, k, and a list of 3 chunks (text + metadata showing which PDF each chunk came from).
Full RAG pipeline	/v1/answer_with_relevant_chunks	query, k=3, model_name, temperature, top_p, max_tokens	JSON with the generated answer, the supporting chunks, and all parameter values used.

Table 4. The two inference tests and their expected results.

Both tests use the `requests` library to send a POST with a JSON payload. The notebook first prints the raw Response object; a `<Response [200]>` confirms the backend processed the request successfully. It then prints the full JSON body so you can inspect the answer text, the chunk contents, and the metadata.

The retrieval-only test is useful for debugging: if the chunks are irrelevant, the problem lies in the embedding model or the chunking strategy, not in the LLM. The full-RAG test validates the entire pipeline end to end.

9. Quick Reference: Why Each Piece Exists

Component	Why It Exists
faiss_index/	Stores pre-computed embeddings so the backend does not need the original PDFs, the Docling converter, or the time-consuming embedding step.
rag_utils.py	Separates the RAG logic (retrieve, generate, full pipeline) from the web-server code, keeping both files clean, testable, and independently maintainable.
app.py (Flask)	Turns the RAG pipeline into a network-accessible REST API with well-defined JSON endpoints that any HTTP client (including the Streamlit frontend) can call.
requirements.txt (backend)	Pins exact library versions to guarantee the same software stack in every build, preventing subtle compatibility bugs.
Dockerfile (backend)	Packages everything (OS, libraries, model weights, FAISS index, application code) into a single portable image that runs anywhere Docker is available.
app.py (Streamlit)	Gives hotel staff a simple, browser-based interface to query the RAG system without any technical knowledge.
requirements.txt (frontend)	Keeps the frontend image lightweight and fast to build: only Streamlit and the requests library are needed.
Dockerfile (frontend)	Runs the Streamlit UI on its own port, fully isolated from the backend, so each service can be updated independently.
GitHub repository	Central, version-controlled store for all deployment files. Codespaces pulls from this repo to build images automatically.
Codespace secrets	Injects OpenAI API keys as environment variables so they never appear in source code, commit history, or container logs.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Component	Why It Exists
Port forwarding	Gives each container a public URL so the notebook (or any browser) can reach the running services from outside Codespaces.

Table 5. Every deployment component and the reason it is needed.

Important: After you have finished testing, delete the Codespace to stop consuming resources. Refer to the separate document titled 'Deleting Codespace' for step-by-step instructions.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.